REQ3 Design Rationale

The Consumable Interface

Before we had REQ3, the way that Apple and Cookies worked was inside their action classes. This was a problem for the Slime because it eats things directly from the ground. The Slime could not use the actions as Apple and Cookies. If we copied the code it would be bad because we would have the same code in two places. This is something we want to avoid.

So we made an interface. This interface has two parts: consumedBy and isExhausted. Each item has its effect logic inside consumedBy. The thing that uses the item like a worker or a slime just calls consumedBy. Then handles removing the item. The worker removes the item from the inventory. The slime removes it from the ground. This way the item does not need to know where it came from.

Getting Items for Ground-Item Discovery

We have a way to get items from the ground called getItemsAs. This uses something called GameEntity.asCapability under the hood. It checks if an item is a type. Since Consumable is an interface any item that uses it will be returned. The EatBehaviour uses getItemsAs to find items on the ground without knowing what they are. If we want to add an edible item we just need to make it use the Consumable interface. We do not need to change EatBehaviour or the Slime.

Behaviour Pattern with TreeMap Priority

The Undead and the Slime use something called a TreeMap to store their AI. This TreeMap has a list of behaviours in order of priority. The playTurn function tries each behaviour in order and

stops at the one that works. This makes it easy to see what the priorities are and add behaviours. Both the Undead and the Slime use the WanderBehaviour as a backup. This way we do not have to repeat code.

Worker Ability for Target Identification

The AttackBehaviour could have checked if a target was a ContractedWorker.. This would have been bad because it would have made the AttackBehaviour depend on the ContractedWorker class. Instead the ContractedWorker enables something called the WORKER ability. The AttackBehaviour checks if a target has this ability. This way the Undead will automatically ignore Undead and Slimes because they do not have the WORKER ability. If we add a type of worker it just needs to enable the WORKER ability and the Undead will attack it.

BareHand as IntrinsicWeapon

The engine already has a way to handle hitting someone with a weapon. We made a weapon called BareHand that uses this. The Undead uses the BareHand to attack. It works like it should. The AttackAction just calls the BareHands attack function and checks if the target died. All the math for hitting is already in the engine.

Isconscious Check in NPC playTurn

The engine does not remove actors when they die from things like poison or fire. It only removes them when we tell it to. The PoisonStatus applies to the Slime. Can kill it.. If we do not check if the Slime is conscious at the start of its turn it might try to act even if it is dead. So we check if the Slime is conscious, at the start of its turn and if it is not we remove it from the map.